CS571: Programming Languages

Assignment 2. Due: Friday, Mar. 6, 11:59PM

For this assignment, you need to submit your solution as a single PDF file to **Brightspace**. All submitted work must be your own work.

Problem 1 [10pt] Consider the following class instances in a C++ program:

```
1  static myClass* X;
2  int main()
3  {
4    myClass* Y = new myClass();
5    foo();
6    delete Y;
7    return 0;
8  }
9
10 void foo()
11 {
12   myClass Z;
13   X = new myClass();
14 }
```

a) (5pt) What is the storage allocation (static/stack/heap) for the following objects?

- object A: the pointer `X` static
- object B: the pointer `Y` stack.
- object C: the `myClass` object created at line 4 heap
- object D: the `myClass` object created at line 12 stack
- object E: the `myClass` object created at line 13 heap

b) (5pt) Consider one execution of the program above. The execution trace (a sequence of program statements executed at run time) of this program is `4 5 12 13 6 7`.
For each object above (i.e., object A to E), write down its lifetime (use a subset of execution trace, e.g., `12 13` to represent the lifetime). Assume the lifetime of a heap-allocated object starts from `new` and ends after `delete` and include both `new` and `delete`.

A {4, 5, 12, 13, 6, 7}.

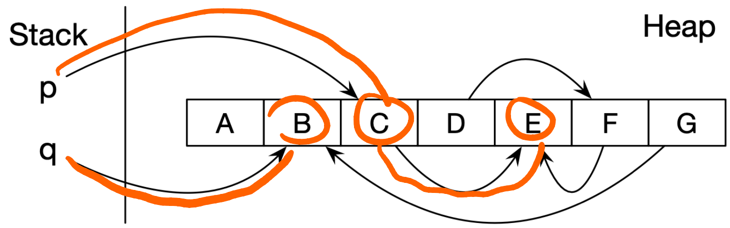
B {4, 12, 13, 6, 7}

C {4, 12, 13, 6}

D {12, 13}

E {13, 6, 7}.

Problem 2 [8pt] Consider the heap state as shown below before garbage collection:



a) (2pt) For the Mark-and-Compact algorithm, what objects remain on the heap after collection? Do they stay in the same location after collection?

remain : B, C, E

not in the same location, they will be moved

b) (6pt) Consider algorithms Mark-and-Sweep, Mark-and-Compact, Stop-and-Copy. For each of them, write down the objects being visited during the collection in sequence. Assume 1) reachability analysis uses depth-first search, which will skip objects that have already been visited, 2) search starts from p, and then q, 3) when the entire heap is traversed, objects are visited from left to right. You don't need to write down the newly created object copies, if any.

Mark-and-Sweep : Mark: C, E, B ; Sweep: A, B, C, D, E, F, G

Mark-and-Compact : Mark: C, E, B ; Compact A B C D E F G

Stop-and-Copy : C E B

Problem 3 [6pt] For each of the following data characteristics, **briefly explain** which of the garbage collection algorithms discussed in class (Mark-and-Sweep, Mark-and-Compact, Stop-and-Copy) would be the most appropriate and why. Make no other assumptions than those given.

a) (3pt) All objects are of the same size and many have relatively long lifetime.

Market-and-Sweep :
∵ obj have same size & long life time.
∴ we don't have to move obj, and copy expensive.

b) (3pt) Most objects have short lifetime and data locality greatly improves application's performance.

Stop-and-copy
∵ life time short
∴ copy won't cost a lot time
also, stop & copy can compact live obj into contiguous memory. which improve data locality.

Problem 4 [14pt] Consider the following pseudo-code, with the assumption that the language uses **static scoping** and has one scope for each function (including the main function), and it allows nested functions (hence, nested scopes).

```
main() {
  int g;
  int x = 4;
  function B(int a) {
    int x = a + 2;
    C(1);
  }
  function A(int n) {
    g = n;
  }
  function C(int m) {
    print x;
    x = x / 2;
    if (x > 1)
      C(m + 1);
    else
      A(m);
  }
  // body of main
  B(1);
  print g;
}
```

a) (5pt) Draw the symbol table tree with 4 tables, one table for each of the 4 scopes (namely `main`, `B`, `A`, `C`) in this program. Assume each table contains two columns: name and kind (e.g, id, fun, para).

next page.

b) (3pt) What does the program print?

4
2
2

c) (4pt) Draw a picture of the run-time stack when function `A` has just been called. For each frame, show the static and dynamic links. You do not have to show the storage for any other information.

next page

d) (2pt) Refer to the run-time stack, briefly explain how function `A` finds variable `g`.

A follows its static link to the activation record of main, where g is declared, and accesses g there.

(a)

main

name	kind
g	id
x	id
B	func
A	func
C	func.

B

name	kind
a	para
x	id

C

name	kind
m	para

A

name	kind
n	para

(c).

Main	g, $x = 4$, B(1).
B	$a = 1$, x(local) $= (a+2) = 1+2 = 3$. C(1)
C	$m = 1$, print ($x = 4$). $x = x/2 = 4/2 = 2$.
C'	$m = 2$ print ($x = 2$) $x = 2/2 = 1$
A	$m = 2$ $g = 2$

Problem 5 [12pt] Consider the following pseudo-code with higher-order function support. Assume that the language has one scope for each function, and it allows nested functions (hence, nested scopes). In the pseudo code, declarations with type `int` are local declarations; otherwise the variables not declared locally are referring to the outer level of function.

```
function A() {
  int x = 5;
  function C(P) {
    int x = 3;
    P();
  }
  function D() {
    print x;
  }
  function B() {
    int x = 4;
    C(D);
  }
  B();
}
A();
```

a) (3pt) What would the program print if this language uses dynamic scoping and shallow binding (binding happens when the function is called)?

print 3

b) (6pt) Justify your answer by showing the tree of symbol tables when execution reaches the `print` expression.

A	$x=5$ B().
B	$x'=4$ C(D)
C	P=D $x'=3$
D	print x

c) (3pt) What would the program print if this language uses static/lexical scoping?

print 5.